GUIDE D'UTILISATION AAM O82_2

ROCA-D / AAM

6 octobre 2026

Table des matières

1 Description **2**

2 Architecture **3**
2.1 Observation dégradée . . . 3
2.2 Mémoire . . . 3
2.3 Apprentissage . . . 3
2.4 Réponse généralisée . . . 3
2.5 Auto-observation . . . 4

3 Pré-requis PC **5**

4 Installation PC **6**

5 Gouvernance des erreurs **7**

6 Créer un épisode **8**

7 Observation **9**

8 Acquisition et apprentissage **10**
8.1 Apprentissage itératif . . . 10

9 Réutiliser une compétence **11**

10 Composition, découverte et décomposition **12**

11 Instruments et auto-observation **13**

12 Mémoire, intégrité et recovery **14**

13 Validation et benchmark **15**

14 Commandes complètes O81 conservées **16**

15 Données texte et langues **17**

16 Dépannage **18**

17 Reproductibilité **19**

18 Statut **20**

Chapitre 1

Description

Ce guide décrit l'installation, l'exécution, l'apprentissage, l'observation, la mémoire, la vérification, la généralisation, l'auto-observation et la récupération de la version AAM O82_2. Le modèle est non-LLM, standard-library-first et exécutable localement. Aucun service externe n'est obligatoire pour le chemin de base.

Le modèle ne contient pas de réponse métier pré-écrite. Les exemples fournis servent uniquement à vérifier l'installation et la chaîne d'exécution. Pour une compétence réelle, les données doivent être acquises par le protocole d'apprentissage.

Chapitre 2

Architecture

Le pipeline général est :

problème + contrat + budget
→ OBSERVE → empreinte partielle
→ retrieval mémoire → suffisance
→ VERIFY → MEMORIZE → ACT/DONE.

Lorsque la preuve n'est pas suffisante, la politique sélectionne une nouvelle observation, un bundle, une correction, une décomposition ou une opération d'apprentissage. En mode expérimenté, une candidate généralisée peut être produite sans devenir automatiquement certifiée.

2.1 Observation dégradée

La mémoire est consultée avant chaque choix de sonde. Lorsqu'une compétence indique les vues minimales nécessaires, le runtime tente ces probes en priorité et réévalue après chaque observation. Une exploration plus large n'est utilisée que lorsque la mémoire ne suffit pas ou lorsqu'un bundle est justifié par les contraintes de ressources.

2.2 Mémoire

Les raw sont conservés séparément de l'index. Les empreintes, témoins, invalidations et vues peuvent être reconstruits à partir des journaux. Une compétence ne doit pas être promue uniquement parce qu'elle ressemble à une compétence existante.

2.3 Apprentissage

Le learner possède un portfolio de candidats génériques. La sélection s'effectue sur TRAIN seulement, puis le candidat sélectionné est réentraîné sur tout TRAIN. Une variante itérative peut enrichir les données internes avec les erreurs des rondes précédentes. HOLDOUT et OOD restent des jeux de mesure, jamais des jeux de tuning.

2.4 Réponse généralisée

Pour une entrée nouvelle, le runtime peut retourner : décision candidate, confiance, source, skill_id et `certified=false`. Une telle sortie est utile pour l'utilisateur mais ne vaut pas preuve exacte. La confiance est un score gradué ; elle n'est pas présentée comme une probabilité garantie.

2.5 Auto-observation

Le runtime peut observer son propre état, déclencher une transition `SELF_OBSERVE`, réserver une ressource pour une action prédite et journaliser une intervention causale. Une configuration désactivée ne doit pas programmer de SELF_OBSERVE automatique.

Chapitre 3

Pré-requis PC

Un PC courant avec Python 3.10+ est suffisant pour le chemin sans installation. Aucun GPU n'est requis. Le mode sans installation est recommandé pour l'essai initial, car il ne dépend pas d'un téléchargement de paquet.

Chapitre 4

Installation PC

Depuis la racine du dossier extrait :

```
cd Dmodel_AAM_O82\_2
python --version
PYTHONPATH=src python -m aam_o82 audit
PYTHONPATH=src python -m aam_o82 doctor
```

L'installation editable facultative est :

```
python -m pip install -e .
```

Si l'environnement est hors ligne et ne possède pas les outils de build nécessaires, utiliser directement le chemin `PYTHONPATH=src` ; le modèle n'en dépend pas.

Chapitre 5

Gouvernance des erreurs

Avant chaque exécution, consulter le répertoire permanent des checklists fourni par le projet. Les erreurs liées au modèle sont conservées dans `checklist_erreurs_model.json` ; les problèmes de ressources, d'archivage ou de banc dans `checklist_erreurs_no_model.json`. Une entrée `open` bloque l'exécution contrôlée tant que le correctif n'a pas été préparé et validé.

Chapitre 6

Créer un épisode

Un épisode associe un problème, un contrat, un budget, un mode et éventuellement une configuration. Exemple minimal :

```
PYTHONPATH=src python -m aam_o82 run \\
  --problem examples/task.json \\
  --contract examples/echo_contract.json \\
  --config config/runtime.json \\
  --workdir runs/runtime \\
  --max-steps 32
```

Le résultat JSON indique le statut terminal, la décision éventuellement certifiée, la preuve, le budget et l'identifiant d'épisode.

Chapitre 7

Observation

Pour ajouter une observation à un épisode existant :

```
PYTHONPATH=src python -m aam_o82 observe \\
  --episode ID \\
  --probe structure \\
  --config config/runtime.json \\
  --workdir runs/runtime
```

La reprise réhydrate les observations déjà persistées et ne recrée pas un nouvel `INIT`.

Chapitre 8

Acquisition et apprentissage

Acquisition simple :

```
PYTHONPATH=src python -m aam_o82 acquire \\
  --dataset examples/demo_dataset.jsonl \\
  --contract examples/echo_contract.json \\
  --config config/acquisition.json \\
  --mode EXPERIMENT \\
  --workdir runs/acquire
```

Le protocole produit une capsule candidate ou vérifiée selon les contrats. Les erreurs et corrections sont journalisées. Le learner généralisant peut comparer plusieurs candidats et refitter le meilleur sur TRAIN complet.

8.1 Apprentissage itératif

Le protocole itératif travaille sur des rondes internes : apprentissage, calcul des erreurs internes, enrichissement, nouveau test, arrêt par stagnation ou limite de rondes. HOLDOUT/OOD sont exclus de la décision d'arrêt du learner.

Chapitre 9

Réutiliser une compétence

Vérifier une compétence sur un holdout :

```
PYTHONPATH=src python -m aam_o82 verify-skill \\
  --skill skill.json \\
  --dataset examples/demo_dataset.jsonl \\
  --split holdout \\
  --contract examples/echo_contract.json
```

Transfert :

```
PYTHONPATH=src python -m aam_o82 transfer \\
  --skill skill.json \\
  --input input.json \\
  --contract examples/echo_contract.json
```

Une entrée exacte peut produire une décision certifiable si ses preuves sont valides. Une entrée nouvelle peut produire une `CANDIDATE` avec confiance et `certified=false`.

Chapitre 10

Composition, découverte et décomposition

Composition :

```
PYTHONPATH=src python -m aam_o82 compose \\
  --skills skill_a.json skill_b.json \\
  --contract examples/echo_contract.json
```

La composition exige des compétences compatibles et vérifiées. La découverte de probe et la décomposition sont soumises au budget, à la vérification et au root-retest.

Chapitre 11

Instruments et auto-observation

Les instruments spécialistes sont des observateurs externes avec contrat, provenance, coût, incertitude et capacité de parallélisation. Un désaccord peut déclencher une auto-observation causale.

```
PYTHONPATH=src python -m aam_o82 experiment \\
  --problem examples/task.json \\
  --contract examples/echo_contract.json \\
  --config config/runtime.json \\
  --workdir runs/experiment
```

La configuration `self_observation.enabled=false` désactive la programmation automatique de SELF_OBSERVE.

Chapitre 12

Mémoire, intégrité et recovery

Vérification mémoire :

```
PYTHONPATH=src python -m aam_o82 memory-verify \\
  --memory runs/runtime/memory
```

Audit :

```
PYTHONPATH=src python -m aam_o82 audit
```

Replay :

```
PYTHONPATH=src python -m aam_o82 replay --episode ID --workdir runs/runtime
```

Le replay doit pouvoir reproduire l'ordre des événements et détecter une altération de chaîne.

Chapitre 13

Validation et benchmark

```
PYTHONPATH=src python -W error::ResourceWarning -m unittest discover -s tests
PYTHONPATH=src python scripts/validate_o82.py \
  --output reports/P11_validation.json
PYTHONPATH=src python scripts/benchmark_o82.py \
  --output reports/P12_benchmark.json
PYTHONPATH=src python scripts/lint_reports.py \
  --output reports/P10_report_lint.json
```

Les avertissements de ressources sont traités comme erreurs dans le gate final.

Chapitre 14

Commandes complètes O81 conservées

La version actuelle conserve aussi les commandes `experiment`, `verify-skill`, `transfer`, `compose` et `memory-verify`. Les adaptateurs externes RAG, firewall, black-box/subprocess, power et protocoles sont facultatifs mais gouvernés.

Chapitre 15

Données texte et langues

Le modèle accepte un problème dont la valeur peut contenir du texte naturel. Les tests SMS ont démontré l'utilisation d'anglais, français, allemand et de langues OOD multilingues. Les métadonnées de langue ne sont pas fournies comme variable de décision au learner dans les expériences. La sortie de classification est textuelle/symbolique. Ce guide ne revendique pas une génération libre de prose.

Chapitre 16

Dépannage

Une sortie `UNRESOLVED` n'est pas une preuve d'impossibilité. Vérifier d'abord contrat, budget, visibilité des probes, preuves disponibles, skill importé et journal de l'épisode. Une exception Python ou un chemin absent doit être enregistré comme erreur de banc avant toute nouvelle exécution.

Chapitre 17

Reproductibilité

Conserver le hash des données, le contrat, la configuration, les paramètres de budget, l'identifiant de skill, les sorties JSON et le journal d'événements. Ne jamais utiliser HOLDOUT/OOD pour choisir un hyperparamètre sans l'indiquer explicitement comme une expérience exploratoire séparée.

Chapitre 18

Statut

Cette version est distribuée avec son modèle normatif, ses tests, ses rapports, ses configurations, ses exemples et ses checksums. Les performances annoncées dans les rapports sont attachées aux protocoles et aux jeux de données mesurés ; elles ne doivent pas être transformées en promesse générale.